AI Service Navigator

Frontend handoff specification – voice assistant for finding the right Caritas support service (DE / EN)

Version	Status	Audience	Backend
0.1 (prototype)	Ready for implementation	Frontend team	FastAPI, local, open-source stack

1. Goal

Build a web page where a person describes their situation by speaking, in everyday German or English, and is guided to the one support service in the regional network that fits best, with a short explanation of why. Guiding principle: **no wrong door to support**. The assistant never gives counseling; it only routes.

The page must answer one question: *“I need help. Which service can support me, and why is it the right place for my situation?”*

2. Scope

In scope	Out of scope
• Single static page: HTML, CSS and vanilla JS (no build step) • Push-to-talk recording and spoken replies • Transcript, service card, DE/EN toggle, text fallback • Accessibility features listed in section 8	• Login, accounts, stored user data • Phone/telephony integration • Admin UI for editing services • Streaming audio / WebRTC (v2)

3. User flow

- User opens the page and sees a short welcome and one large button.
- User presses and holds (or taps) the button and speaks. Language is auto-detected.
- Frontend sends the audio to the backend and shows a “thinking” state.
- Backend returns either a **follow-up question** (spoken and shown) or a **recommended service** with a reason.
- If a follow-up was asked, the user answers the same way; the conversation continues (usually 1–3 turns).
- When a service is recommended, the frontend shows the service card and speaks the reason.
- User can start over at any time.

4. Screen layout and components

Component	Requirements
Header	App name, DE/EN toggle (persist choice in `localStorage`, wrapped in try/catch), text-size control (A− / A+).
Notice banner	Always visible, plain language. DE: “Ich gebe keine Beratung, ich helfe Ihnen nur, die richtige Stelle zu finden.” EN: “I don’t give counseling. I only help you find the right service.” Emergency line: 112; Telefonseelsorge 0800 111 0 111.
Talk button	Large (min. 96 px), centered. Hold-to-talk **and** tap-to-toggle. States: idle, listening, thinking, speaking, error. Shows icon, label and color change per state (never color alone).

Component	Requirements
Conversation	Chat-style list: user bubbles (transcript) and assistant bubbles. Auto-scroll to latest. Assistant bubble has a replay button.
Service card	Appears when `service` is returned: name, “Why this fits you” (1–2 sentences), address, phone (tel: link), email (mailto: link), website, opening hours, languages offered. Highlighted; keeps previous recommendations in history.
Text fallback	Single-line input plus send button. Same request path as voice, using the `text` field.
Start over	Clears history and session ID; stops any playing audio.

5. UI state machine

State	Entered when	UI behavior	Next
idle	Page load, after reply finished	Button reads “Sprechen / Speak”	listening
listening	Button pressed, mic granted	Pulsing ring, timer, live level meter if available	thinking (on release/stop)
thinking	Request sent	Spinner and “Ich höre zu… / Thinking…”; button disabled	speaking or error
speaking	Reply audio playing	Stop button; reply text visible	idle
error	Mic denied, network fail, empty audio, HTTP error	Friendly message and Retry; text fallback stays usable	idle

Cancel rule: pressing the button while the assistant speaks must stop playback and start a new recording (barge-in).

6. API contract

Base URL configurable via a single constant (`API_BASE`), default `http://localhost:8000`. CORS is enabled on the backend.

POST /talk

Content type `multipart/form-data`. Send either `audio` or `text`, not both.

Field	Type	Notes
audio	file (webm/opus or wav)	From MediaRecorder. Max 30 s.
text	string	Used by the text fallback.
language	“auto” | “de” | “en”	“auto” unless the user set the toggle.
session_id	string (UUID)	Created client-side on load; reset on Start over.

Response 200 (application/json)

```
{
  "session_id": "b1f3…9c",
  "transcript": "Ich weiß nicht mehr weiter, meine Mutter ist pflegebedürftig",
  "detected_language": "de",
  "reply_text": "Pflegen Sie Ihre Mutter zu Hause?",
  "reply_audio_url": "/audio/7a2c.wav",
  "needs_more_info": true,
  "service": null
}
```

When a recommendation is ready, `needs_more_info` is false and `service` is set:

```
"service": {
  "id": "svc_017",
  "name": "Pflegeberatung für Angehörige",
  "why": "Sie pflegen Ihre Mutter zu Hause. Diese Stelle berät pflegende Angehörige.",
  "address": "Musterstraße 1, 97070 Würzburg",
  "phone": "+49 931 000000",
  "email": "info@example.org",
  "website": "https://example.org",
  "opening_hours": "Mo–Fr 9–12 Uhr",
  "languages": ["de", "en"]
}
```

Other endpoints

Endpoint	Purpose
GET /audio/{id}.wav	Fetch the TTS reply. Play with an <audio> element or Web Audio.
GET /health	Returns {“ok”: true}. Call on load; show a banner if it fails.

Errors: non-2xx returns `{"error": "code", "message": "..."}`. Codes: `empty_audio`, `no_speech`, `server_busy`, `internal`. Map each to a friendly DE/EN message.

7. Languages and copy

- All UI strings live in one `i18n` object with de and en keys. No hard-coded text in markup.
- Default UI language: from `navigator.language`, German if unknown. The toggle always wins over detection.
- Set `<html lang>` to match the active language, so screen readers pronounce text correctly.
- Use plain, short sentences (target reading level A2–B1). No jargon, no menus, no category pickers.
- Service data is returned in German; `reply_text` and `why` are already in the user’s language.

8. Accessibility requirements

Area	Requirement
Contrast and size	WCAG 2.1 AA contrast. Base font 18 px or larger; A− / A+ scales up to 200%. Touch targets at least 48 px.
Keyboard	Everything reachable by Tab. **Space** toggles recording on the talk button. Visible focus ring. Esc stops playback.
Screen readers	Conversation list is an `aria-live="polite"` region. Button has `aria-pressed` and a state-dependent `aria-label`. Service card is a labelled region with a heading.
Non-audio users	Every spoken reply is also shown as text. Text input fallback always present.
Motion	Respect `prefers-reduced-motion`: disable pulsing and animations.
Theming	Light and dark mode via `prefers-color-scheme`, with explicit background color on body.

9. Browser and technical notes

- Mic access (`getUserMedia`) requires HTTPS or `localhost`. Demo runs on localhost.
- Recording: `MediaRecorder` with `audio/webm;codecs=opus`; fall back to the default MIME type on Safari. Always stop all tracks after recording.
- Handle denied or unavailable mic with a clear message and the text fallback.
- CPU-only backend can take 4–8 s per turn. The thinking state is mandatory; set a request timeout of 60 s with a Retry.
- Minimum targets: current Chrome, Edge, Firefox, Safari (desktop) and Chrome / Safari on mobile at 360 px width and up.

- No external CDN or font dependencies, so the page runs offline on the local network. Open-source libraries only.

10. Acceptance criteria

#	Criterion
1	A user can speak a German request and receive a spoken and written follow-up question or recommendation.
2	The same works in English, and the DE/EN toggle changes all UI text immediately.
3	A recommended service shows name, reason, and at least one working contact link (tel, mailto or website).
4	Multi-turn: the second answer is sent with the same session_id and the backend uses prior context.
5	Every error state in section 5 shows a friendly message and the user can recover without reloading.
6	The whole flow works with keyboard only, and with a screen reader announcing replies.
7	The notice banner (no counseling, emergency numbers) is visible on every screen state.
8	No horizontal scroll at 360 px width; no console errors during a full run.

11. Deliverables and open points

- `index.html`, `styles.css`, `app.js` (or a single file), plus a short README with run instructions.
- A mock mode (`?mock=1`) that returns canned responses so the UI can be built without the backend.
- Open: final visual identity (logo, colors). Use neutral defaults until the NGO provides branding.
- Open: final service data fields may grow (e.g. target group, cost, appointment needed). The service card should render unknown extra fields gracefully.